



Hardenix

Auditor y endurecedor de seguridad para **Linux**. Audita, puntúa y **corrige** tu sistema — con copia de seguridad y rollback.

- Auditoría
- Auto-fix + rollback
- Informe HTML
- IA local
- Dashboard web
- Multi-distro

 **Hardenix · Dashboard**

Kali GNU/Linux Rolling · 2026-06-07T13:38:56

Ejecutar auditoría

PUNTUACIÓN

92

/ 100

21 OK

2 fallos

0 n/a

EVOLUCIÓN



5 auditorías · última: 92/100

HALLAZGOS

ESTADO	SEV.	COMPROBACIÓN	DETALLE
FALLO	MEDIA	Autenticación por contraseña deshabilitada (solo claves)	Se permite login por contraseña (vulnerable a fuerza bruta). · esperado: PasswordAuthentication no
FALLO	MEDIA	auditd instalado y activo	Instala el paquete: apt install auditd · esperado: auditd instalado y activo
OK	CRÍTICA	Ninguna cuenta con contraseña vacía	
OK	ALTA	Login de root por SSH deshabilitado	
OK	ALTA	Contraseñas vacías por SSH deshabilitadas	
OK	ALTA	ASLR activado (randomize_va_space = 2)	
OK	ALTA	Solo la cuenta root tiene UID 0	
OK	ALTA	Permisos correctos en /etc/shadow	
OK	ALTA	Permisos correctos en /etc/gshadow	
OK	ALTA	Cortafuegos activo	
OK	ALTA	Servicios inseguros heredados deshabilitados (telnet, rsh, tftp...)	
OK	MEDIA	MaxAuthTries limitado (<= 4)	
OK	MEDIA	TCP SYN cookies activadas	
OK	MEDIA	ICMP redirects no aceptados	
OK	MEDIA	Algoritmo de hash de contraseñas fuerte	

¿Qué es y qué problema resuelve?

Endurecer ("bastionar") un servidor Linux es una de las tareas básicas de seguridad: desactivar accesos peligrosos, ajustar el kernel, cerrar servicios inseguros... El problema es que la mayoría de herramientas **solo te avisan** de lo que está mal, pero no lo arreglan ni te dejan medir la mejora.

La idea de Hardenix

Un ciclo completo y medible: **auditar** → **puntuar** → **corregir** → **volver a puntuar**, con la red de seguridad de poder deshacer cualquier cambio.

Corrige, no solo avisa

Aplica las correcciones y **respalda cada cambio**; un comando lo revierte todo.

Puntuación y antes/después

Una nota de 0 a 100 ponderada por severidad, e informe comparativo de la mejora.

IA local y privada

Explica cada fallo en español con un LLM en tu propia máquina (LM Studio). Nada va a la nube.

Multi-distro

Detecta la familia (Debian/Ubuntu y RHEL/Fedora) y adapta comprobaciones y comandos.

Comprobaciones (cerca de 30, estilo CIS Benchmark)

CATEGORÍA	EJEMPLOS
SSH	login de root, solo-claves, MaxAuthTries, LoginGraceTime, StrictModes, X11
Kernel (sysctl)	ASLR, SYN cookies, ICMP redirects, reverse path filtering
PAM	complejidad (pwquality), historial (pwhistory), bloqueo por fuerza bruta (faillock)
Cuentas	caducidad de contraseña, UMASK, algoritmo de hash, UID 0 único
Permisos	/etc/shadow, /etc/gshadow, /etc/passwd, /etc/group
Red / Servicios	cortafuegos activo; telnet/rsh/tftp deshabilitados
Auditoría / Parches	auditd activo; actualizaciones de seguridad pendientes

El flujo: auditar, corregir, verificar

Hardenix evalúa cada control de seguridad de forma independiente (módulos), calcula una puntuación y, para los fallos, ofrece una corrección reversible.

1 · Auditar

Lee la configuración real del sistema (sshd, sysctl, login.defs, permisos, servicios...) y marca cada control como **OK** o **FALLO**.

2 · Puntuar

Nota 0–100 ponderada por severidad (crítica > alta > media > baja).

3 · Corregir (con rollback)

Aplica los fixes y guarda un *snapshot* de cada fichero/valor tocado. Seguro por defecto: solo previsualiza salvo `--yes`, y omite los fixes que podrían bloquear el acceso (p. ej. SSH solo-claves) salvo que se pidan explícitamente.

4 · Volver a puntuar

Re-auditoría automática que muestra el salto de puntuación (p. ej. 78 → 92).

Si una corrección no conviene, `hardenix rollback` restaura el estado anterior (ficheros, valores del kernel y servicios) a partir del snapshot guardado.

Panel con puntuación y evolución

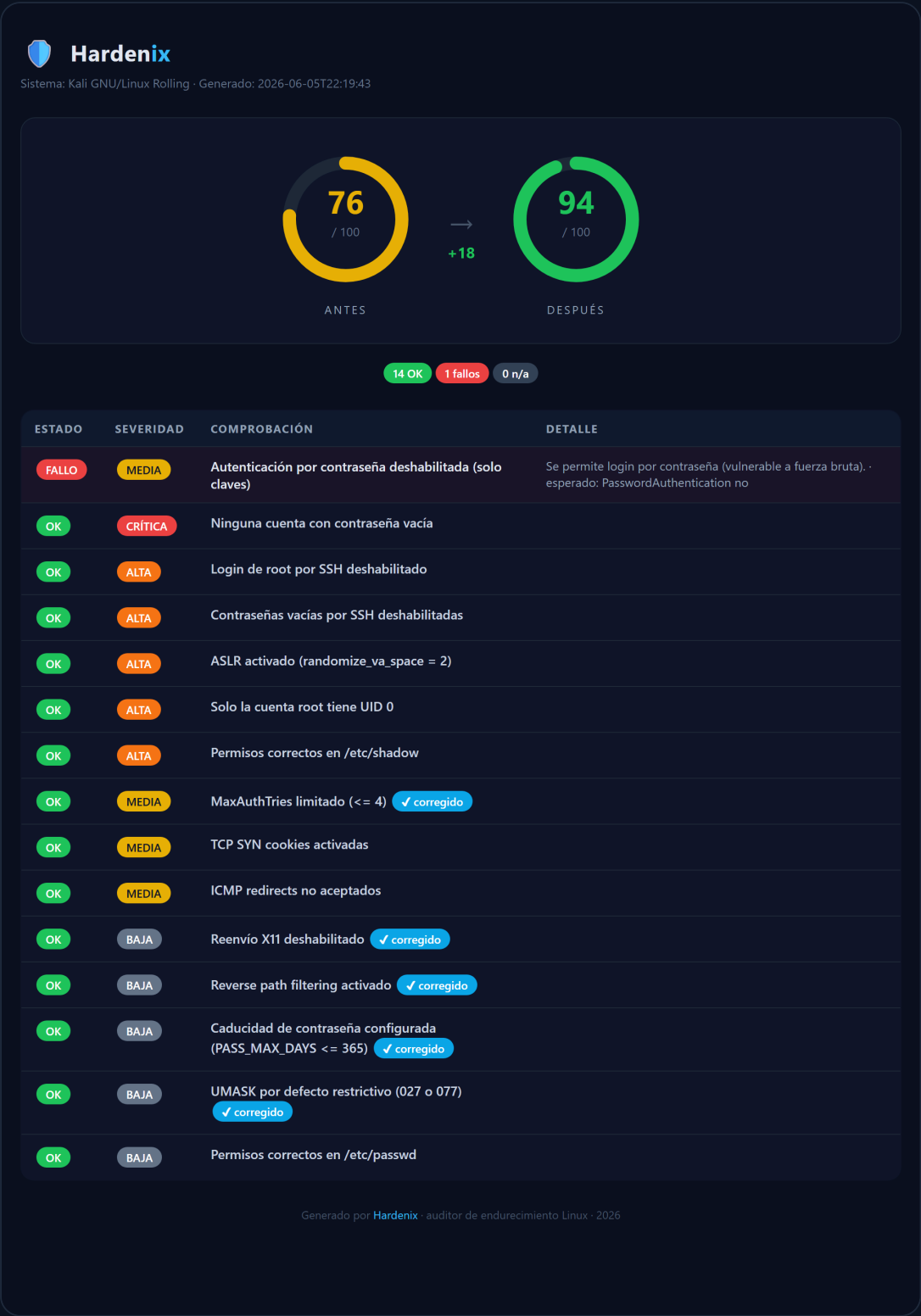
Un dashboard local (servidor de la librería estándar, sin dependencias) con la puntuación en vivo, el historial de auditorías y la **evolución del endurecimiento** en el tiempo.



```
python3 -m hardenix serve → http://127.0.0.1:8080
```

Comparativa antes / después

Informe autocontenido que muestra el salto de puntuación y marca los controles corregidos.



Explicaciones con IA privada

Con --ai, un LLM local (LM Studio) explica cada hallazgo en español: qué es, por qué importa y el riesgo real. Todo el procesamiento ocurre en tu máquina.

**Hardenix**

Sistema: Kali GNU/Linux Rolling · Generado: 2026-06-05T22:56:53

71

/ 100

PUNTUACIÓN

8 OK

6 fallos

1 n/a

ESTADO	SEVERIDAD	COMPROBACIÓN	DETALLE
FALLO	MEDIA	Autenticación por contraseña deshabilitada (solo claves)	Se permite login por contraseña (vulnerable a fuerza bruta). · esperado: PasswordAuthentication no El hallazgo de autenticación por contraseña deshabilitada y solo claves significa que el acceso al servidor se realiza únicamente mediante llaves SSH, eliminando la posibilidad de usar contraseñas. Esto es importante porque las contraseñas son vulnerables a ataques de fuerza bruta, lo cual podría permitir que un atacante acceda al sistema si logra adivinar o probar suficientes combinaciones. Deshabilitar la autenticación por contraseña aumenta la seguridad del servidor al reducir el riesgo de intrusiones.
FALLO	MEDIA	MaxAuthTries limitado (<= 4)	· esperado: MaxAuthTries 4 Este hallazgo indica que el número máximo de intentos de autenticación permitidos en un sistema Linux ha sido limitado a 4, lo cual es una práctica de seguridad para reducir los ataques de fuerza bruta. Si no se corrige y mantiene la configuración original (MaxAuthTries 6), aumenta el riesgo de que un atacante pueda probar múltiples contraseñas sin restricciones, lo cual es crucial para proteger las cuentas críticas del sistema.
FALLO	BAJA	Reenvío X11 deshabilitado	· esperado: X11Forwarding no El reenvío X11 es una característica que permite la transferencia de ventanas gráficas entre clientes y servidores. Deshabilitarlo mejora la seguridad al evitar la exposición accidental o intencionada a sistemas remotos, especialmente en configuraciones no seguras. Si este parámetro permanece habilitado, aumenta el riesgo de ataques que puedan aprovechar esta vulnerabilidad para obtener acceso no autorizado a los servidores X11.
FALLO	BAJA	Reverse path filtering activado	El rp_filter descarta paquetes con origen suplantado (anti-spoofing). · esperado: net.ipv4.conf.all.rp_filter = 1 Reverse path filtering, o filtrado de ruta inversa, es una medida de seguridad que verifica la autenticidad del origen de un paquete antes de permitir su entrada en el sistema. Si está desactivado (como en este caso con 'net.ipv4.conf.all.rp_filter = 0'), puede permitir paquetes con direcciones IP falsas, lo cual es inseguro. Recomendar activarlo (con 'net.ipv4.conf.all.rp_filter = 1') mejora la seguridad contra ataques de suplantación de identidad, reduciendo el riesgo de intrusiones y otros tipos de vulnerabilidades relacionadas con direcciones IP falsas.
FALLO	BAJA	Caducidad de contraseña configurada (PASS_MAX_DAYS <= 365)	· esperado: PASS_MAX_DAYS 365 La caducidad de contraseñas es una práctica de seguridad que obliga a los usuarios a cambiar sus contraseñas regularmente. Esto reduce el riesgo de compromiso si alguna contraseña se ve afectada, ya que obliga a los usuarios a cambiar su contraseña periódicamente ayuda a mitigar la exposición prolongada de una sola contraseña. En este caso, el hallazgo indica que actualmente las contraseñas pueden mantenerse indefinidamente (PASS_MAX_DAYS 99999), mientras que lo recomendado es establecer un límite máximo de caducidad de 365 días para asegurar una rotación adecuada. La severidad baja indica que el riesgo directo no es alto, pero mantener la práctica recomendada mejora la seguridad general del sistema.
FALLO	BAJA	UMASK por defecto restrictivo (027 o 077)	· esperado: UMASK 027 **Hallazgo**: El valor de UMASK predeterminado es demasiado restrictivo, lo que limita las capacidades de los usuarios y grupos en el sistema. **Importancia**: Un UMASK más permisivo permite a los usuarios y grupos tener acceso directo a nuevos archivos y directorios sin necesidad de permisos adicionales. Esto mejora la eficiencia operativa y la colaboración dentro del equipo. **Riesgo no corregido**: Si el UMASK predeterminado permanece en 027 o 077, los usuarios podrían enfrentarse a limitaciones innecesarias al crear nuevos archivos y directorios, lo que podría ralentizar

Instalación y uso

Escrito en **Python**. La auditoría base **no requiere dependencias** (solo librería estándar). Probado en Debian/Ubuntu y Kali; compatible con RHEL/Fedora.

Instalar

```
git clone https://github.com/Oskiir23/hardenix.git
```

Auditar y corregir

```
# Auditar el sistema
python3 -m hardenix audit

# Ver qué corregiría (vista previa, no cambia nada)
sudo python3 -m hardenix fix

# Aplicar correcciones (respalda cada cambio)
sudo python3 -m hardenix fix --yes

# Deshacer el último conjunto de cambios
sudo python3 -m hardenix rollback --yes
```

Informes y dashboard

```
# Informe HTML con antes/después en un paso
sudo python3 -m hardenix fix --yes --report informe.html

# Explicaciones con IA local
python3 -m hardenix audit --ai

# Dashboard web
python3 -m hardenix serve
```

Proceso de desarrollo

Construido por fases, cada una funcional y verificada en una VM de pruebas, con tests automáticos y CI (GitHub Actions):

FASE	ENTREGA
1	Motor de comprobaciones, puntuación e informe en terminal
2	Corrección automática con copia de seguridad y rollback
3	Informe HTML con comparativa antes/después
4	Explicaciones con IA local (LM Studio)
5	Más controles (firewall, servicios, auditd, PAM, parches) y multi-distro
6	Dashboard web con historial y evolución

Stack técnico

Python

Linux

SSH / sshd

sysctl / kernel

PAM

systemd

CIS Benchmark

http.server

HTML / SVG

LLM local

unittest + CI

Git

Aprendizajes

- Diseño modular y extensible (añadir un control = una clase).
- Cambios reversibles y seguros por defecto: la base de cualquier herramienta que toca producción.
- Integración de IA local manteniendo la privacidad de los datos.
- Validación con tests automáticos y CI en verde.



Código abierto en GitHub
github.com/Oskiir23/hardenix